Problem 1: Bag-of-Words

1.1 Bag-of-Words Design Decision Description

Our feature representation is a standard Bag-of-Words model implemented using the `CountVectorizer` tokenizer. The goal of the pipeline for Problem 1 is to convert each text into a fixed-length numerical vector representing the word usage. For **Text Cleaning and Tokenization**, we transform each text such that:

1. Split the raw text based on white space.
2. Since we don't specify `CountVectorizer's` punctuation parameter, it removes all basic punctuations.
3. Convert all characters to lowercase.

We avoid cleaning the data further as simple processing avoids unintentional distortions when it comes to interpreting our texts.

As for **Vocabulary Construction**, our vocabulary list is built solely off of our training set for the purposes of minimizing data leakage. We include all tokens that appear in at least `min_df` and at most `max_df` training texts, where `min_df` and `max_df` are hyperparameters. This excludes the rare word cases that our model can't generalize on. Note that each size of our vocabulary varies slightly across folds as it's derived from our training split. Typically, the vocab size is 500 - 1000 (670 for our best case)

For **Feature Representation:**, each text is represented as a count vector, where we have the vocabulary size, and the entry $x_{i,j}$, number of times word j appears in text i. We use raw counts in this instance. For handling out-of-vocabulary words, any token that was not originally in our training data gets ignored.

```
for each training fold:
build vocabulary from training reviews
    tokenize each review
    count document frequency of each token
    keep tokens with df >= min_df and <= max_df

vectorize training reviews into count matrix
train logistic regression model

vectorize validation reviews using same vocabulary
compute validation accuracy
```

1.2 Cross Validation Design Description

We first split the dataset into 80% training data and 20% held-out validation data in order to estimate how well the model generalizes to unseen data. Hyperparameter selection is only performed on the training portion using 5-fold cross validation with shuffling. In this procedure, the training data is partitioned into 5 equal folds, where in each iteration four folds (80%) are using for training and the remaining fold (20%) is used for validation. Shuffling ensures that class labels are more evenly distributed across folds and reduces sensitivity to the original ordering of the dataset.

For our hyperparameter selection, we used `GridSearchCV`. We vary at least 5 distinct hyperparameter combinations, ensuring that we meaningfully explore model complexity rather than implicitly overfitting to a single configuration. For evaluating correctness, we optimize AUROC to measure the model's ability to rank positive examples above negative ones across all classification thresholds. After cross-validation identifies the best-performing configuration, `GridSearchCV` automatically refits the entire pipeline on the full training dataset using those selected hyperparameters. This refit model becomes our final trained classifier. We then evaluate this single final model once on the held-out validation set to obtain an unbiased estimate of performance.

Visualization of this cross validation process:

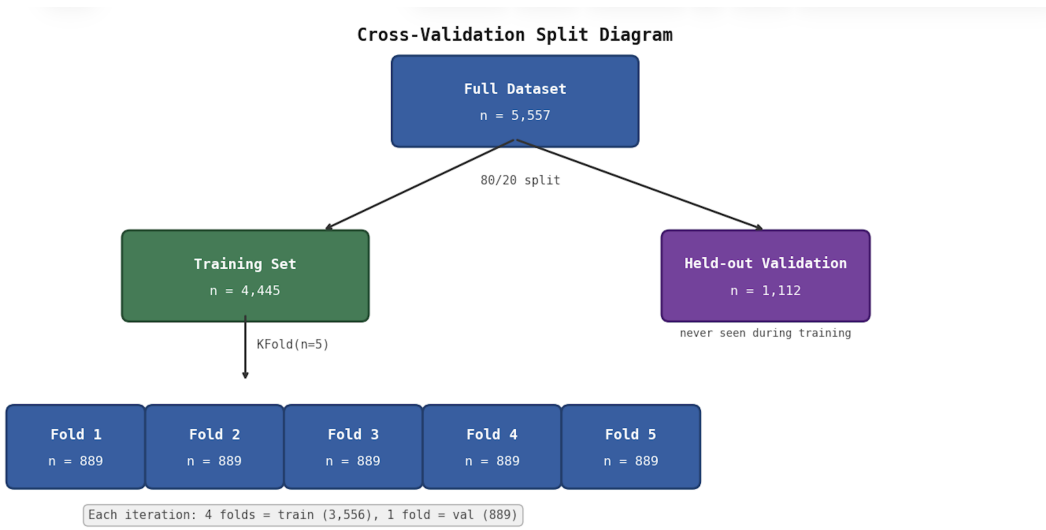

Figure 1: Visualization of Cross Validation Process

1.3 Hyperparameter Selection for Logistic Regression Classifier

We optimize our model over three hyperparameters: C, `min_df`, and `max_df`. This is done with `GridSearchCV` over these values:

Hyperparameter	Values Searched
`min_df`	$\{0.01,\ 0.03,\ 0.05,\ 0.10\}$
`max_df`	$\{0.85,\ 0.90,\ 0.95,\ 1.00\}$
`C`	$\{10^{-4},\ 10^{-3},\ 10^{-2},\ 10^{-1},\ 10^{0},\ 10^{1},\ 10^{2},\ 10^{3},\ 10^{4}\}$

Table 1: Hyperparameter grid used for BOW parameter selection.

The min and max values represent the percentage of the input text that contains a given word. Percentages were used rather than discrete values, as we thought it gave us a better range of potential values across the `min_df` and `max_df` parameters. We also noticed that with a very low `min_df`, the input features that had the largest coefficients ended up being names and other words that shouldn't actually have a large impact on the model.

The regularization parameter C controls model complexity by determining the strength of the L2 penalty applied to the logistic regression weights. Exploring several magnitudes of C allows us to observe the transition between overfitting vs underfitting models.

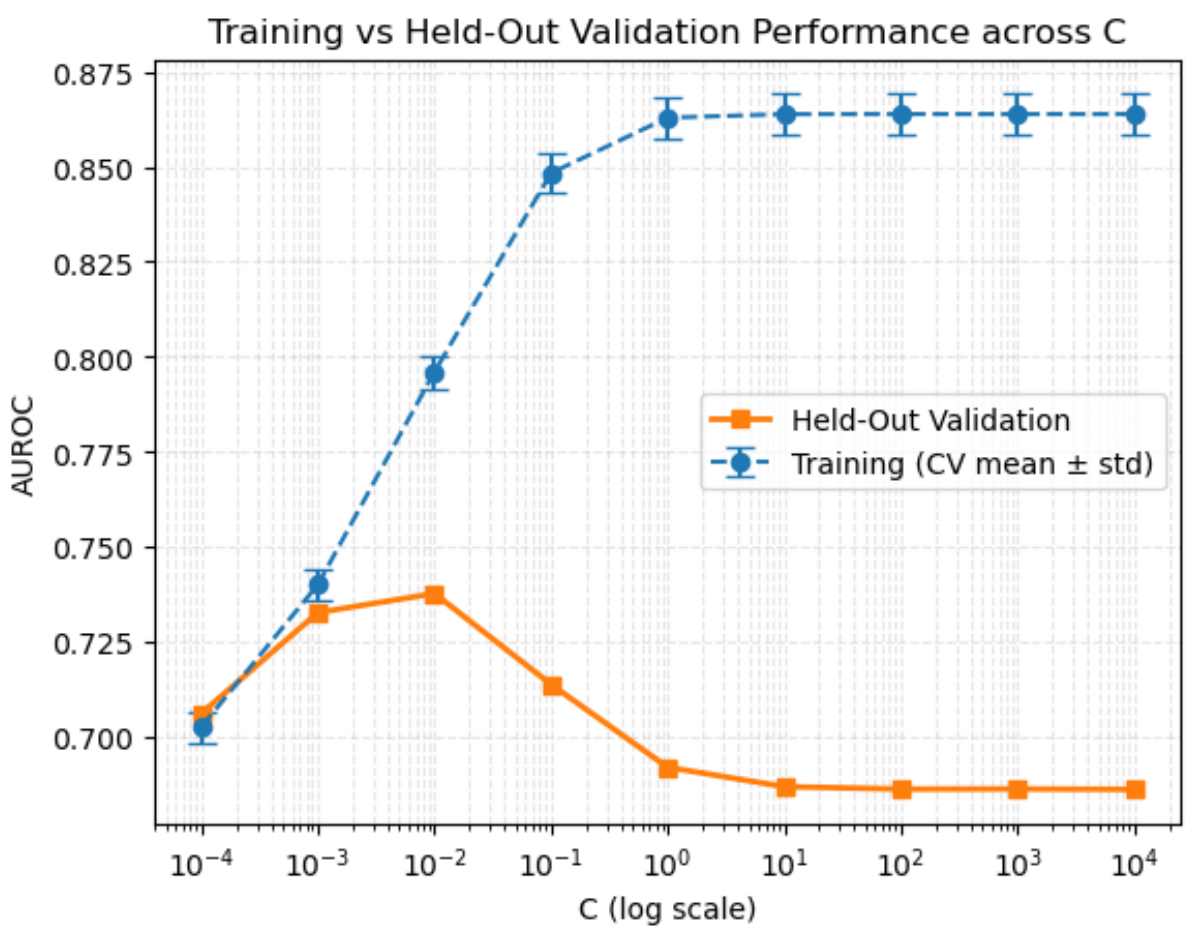

Figure 2: Hyperparameter Search Visualization Across C values

The above graph shows the AUROC scores across the training folds during cross validation, as well as the AUROC score on the held out validation data while varying the hyperparameter C. These values are for when `min_df` and `max_df` are held constant at their best performing values. The plot shows underfitting with small C values, and overfitting as the C value gets higher. The best value of C was $10^{-}2$ as that is where the held out validation data performs the best. Smaller values lead to underfitting due to strong regularization, while larger values reduce the regularization penalty and result in overfitting, as evidenced by increasing training performance but decreasing validation performance.

1.4 Reflecting on Your Test Set Performance

Our AUROC score on the test set is 0.69096.

A large gap between cross-validated performance and test performance suggests that the training folds and the test set are drawn from slightly different distributions, making one generalization task easier than the other. In the context of our training dataset, repeated stylistic patterns or identical phrasings within the same author/source may cause this issue, while the test set contains texts from different stylistic contexts. Since the BoWs model relies on surface word frequencies, they perform well when the same patterns are consistent across our folds. However, if any variation were to occur within the test set, the learned word weights do not transfer as cleanly, leading to our drop in test performance.

To improve performance in Problem 1, we should ensure that CV mimics test conditions more succinctly. Additionally, reducing overfitting through stronger regularization (smaller C) or more aggressive vocabulary pruning (min-df) may help the model rely less on dataset-specific words and more on broadly generalizable signals. Overall, the gap highlights the importance of aligning the validation design with generalizing our procedure across all vocabulary types.

Problem 2: Open Ended Challenge

2.1 Feature Representation Description

For this model, we will be utilizing a combination of the provided numerical features and Word2Vec vector embeddings derived from the text data.

Numerical Features Used Several linguistic and readability-based numerical features were included to capture stylistic differences between each text input. We chose these parameters in hopes of encapsulating structural properties of the passages: The numerical features included:

- `avg_word_length`
- `avg_sentence_length`
- `info_type_token_ratio`
- `readability_FleschReadingEase`
- `readability_DaleChallIndex`
- `punctuation_frequency`

Word2Vec Overview We noticed from a conceptual point of view that `CountVectorizer` isolates each word to vectorize themselves, taking each word out of contexts within the given passages. Our solution was to use `Word2Vec` trained from Google News, an embedding that captures semantic information of texts. This model specifically contains vector representations of 3 million words/phrases, where each word is represented as a 300-dimensional vector.

For each text entry, we converted all words with available embeddings into their corresponding 300-dimensional vectors. These vectors were then combined using a TF-IDF weighted average, which penalizes how common a word pops up in the context of the entire collection. This left us with an individual 300 dimensional vector for each text passage. However, a 300-dimensional vector, combined with just 6 numerical feature parameters, may "wash" out and introduce noise to our model, so to reduce dimensionality and limit redundancy, we implemented Principal Component Analysis to these vectors, reducing them to 20 components. The number of components was determined during our hyperparameter search.

2.2 Cross Validation Description

Our cross validation method was very similar to our bag-of-words model with two note-worthy changes:

1) We opted to not perform an initial train-test split on the data. This was because we wanted to have more data to train on, rather than losing 20 percent of the data as an extra validation check. We decided to transition to this methodology as our training set is already small in terms of training with high-dimensional embeddings; Due to the curse of dimensionality, we require all the data we can obtain for the purposes of training.

2) To ensure the integrity of our evaluation, we partitioned the dataset so that all training texts from a single author were restricted to the same fold. This was designed specifically to prevent **data leakage**, an apparent issue with our BoW model where the overlap of authors in the training + testing sets gave optimistic performance evaluations. This is also a logical

change as the test data contains novel passages, so this better simulates that within our cross validation. We implemented this using the `GroupKFold` class from scikit-learn and specifying our grouping feature as "author".

Visualization of this Cross Validation Process:

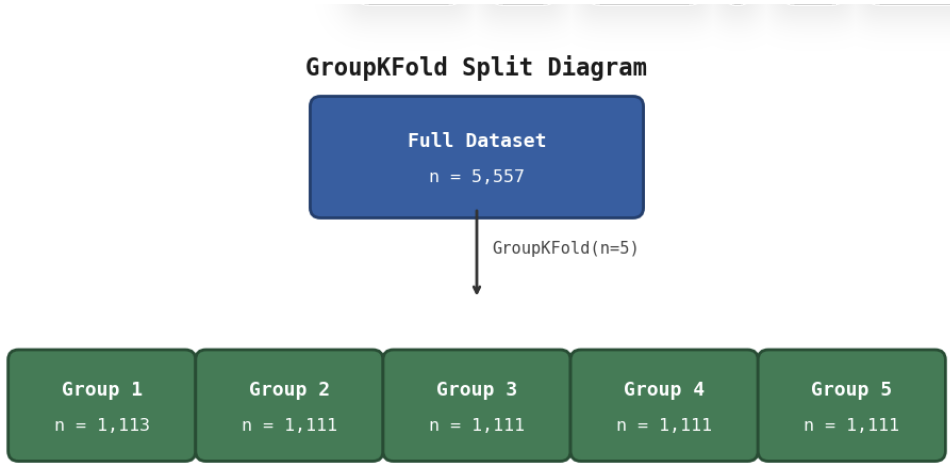

Figure 3: Visualization of Cross Validation Process

As before, after cross-validation identifies the best-performing configuration according to mean AUROC, `GroupKFold` automatically refits the entire pipeline on the full training dataset using those selected hyperparameters. This refit model becomes our final trained classifier.

2.3 Classifier Description With Hyperparameter Search

We again use a logistic regression model, this time with the number of components in Principal Component Analysis as well as the C regularization value as hyperparameters. Logistic Regression on vector embeddings have better interpretability, regularization prevents overfitting metrics, and PCA in a vacuum is efficient on high dimensional data. Hyperparameters are searched using `GridSearchCV` over these values:

Hyperparameter	**Values Searched**
`PCA_n`	$\{20,\ 50,\ 100\}$
`C`	$\{logspace(-4, 4, 9)\}$

Table 2: Hyperparameter grid used for parameter selection.

The number of components in the PCA plays a role in our model complexity as it directly influences the number of features in the model. We chose this range of values as we wanted to significantly reduce the number of components in each vector from the initial 300. PCA allows us to reduce high-dimensional embeddings from `Word2Vec` into fewer, uncorrelated features that preserve maximum variance.

The regularization factor C also significantly impacts model complexity as it adds a penalty to large coefficients. We used the same range of C values from part 1, as when we were plotting our training and validation AUROC over C, we still noticed underfitting as well as overfitting.

Logistic regression training converges reliably using the default solver, and no convergence issues were observed during training. Cross-validation within GridSearchCV was used to ensure robust hyperparameter selection without requiring early stopping.

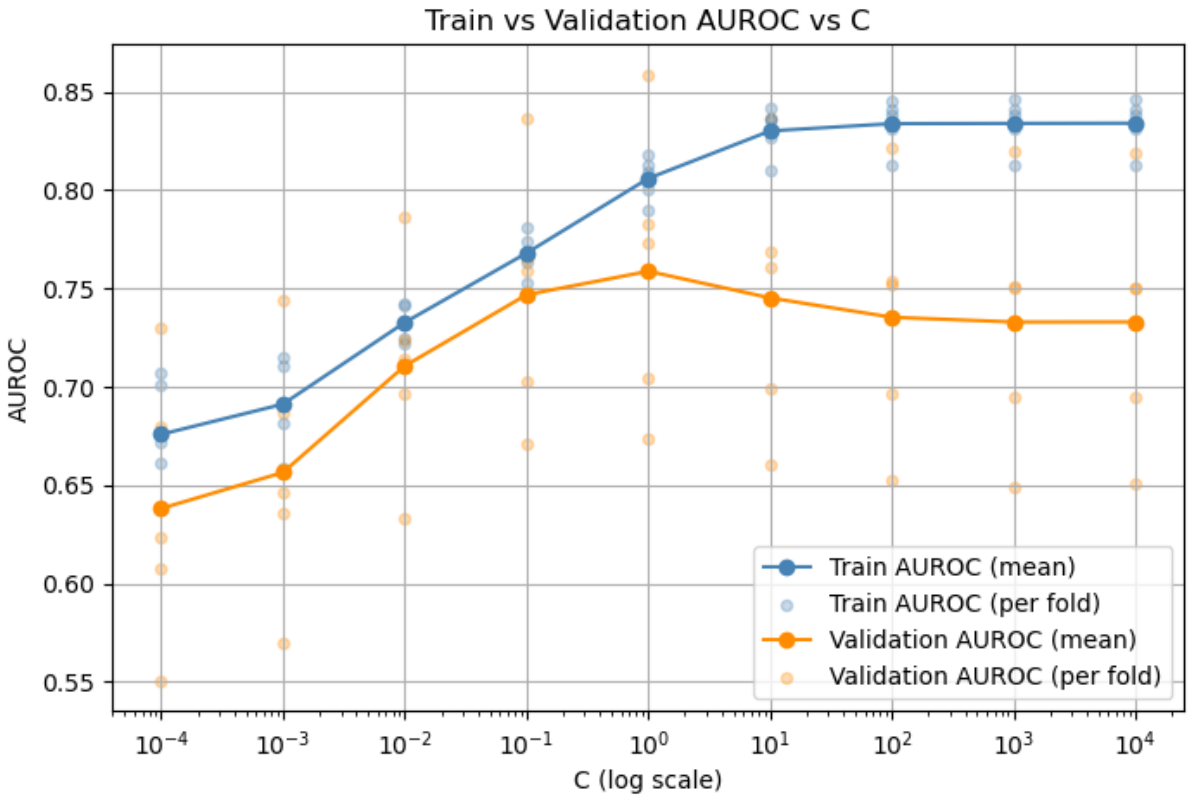

Figure 4: Hyperparameter Search Across C values

The above graph shows the AUROC scores across the training folds and validation folds during cross validation while varying the hyperparameter C. These values are for when `PCA_n` is held constant at its best performing value.

At lower C values, we observe significantly lower AUROC scores for both the training and validation sets. This corresponds to the regularization strength being too high, penalizing the model weights and preventing pattern capturing. The mean AUROC peaks at around 10^0, but the variation in the individual folds suggests that specific C is not necessarily the best hyperparameter. The graph also shows overfitting as the C value gets above 10^1.

2.4 Reflecting on Model Performance

Our best model recorded a 0.71437 AUROC on the test data.

Most Confident Correct Predictions:

Title	**Author**	**True Label**	**Model Pred Proba**
The Ethics of Aristotle	Aristotle	1	0.9998
Discourse on the Method ...	René Descartes	1	0.9997
The Golden Asse	Apuleius	1	0.9982

Most Confident Wrong Predictions:

Title	**Author**	**True Label**	**Model Pred Proba**
Little Brother	Cory Doctorow	0	0.9946
Mrs. Dalloway	Virginia Woolf	0	0.9909
Mrs. Dalloway	Virginia Woolf	0	0.9836

Our Bag-Of-Words model recorded a 0.69096 AUROC score on the test set.
Bag-Of-Words most confident correct and incorrect predictions:

Most Confident Correct Predictions:

Title	**Author**	**True Label**	**Model Pred Proba**
The Golden Asse	Apuleius	1	0.9542
Discourse on the Method ...	René Descartes	1	0.9530
The Red House Mystery	A. A. Milne	0	0.0536

Most Confident Wrong Predictions:

Title	**Author**	**True Label**	**Model Pred Proba**
Oblomov	Ivan Aleksandrovich Goncharov	0	0.8998
Men of Iron	Howard Pyle	0	0.8660
Oblomov	Ivan Aleksandrovich Goncharov	0	0.8600

From the most confidently correct predictions, our 2nd model successfully captures the distinctive vocabulary patterns associated with these texts, perhaps correlating to Euro-centric language. However, the model also makes highly confident mistakes as depicted above, suggesting that the model relies heavily on surface-level word patterns that overlap across classes. We see the same generalization for our BoW approach. Overall, both models capture somewhat relevant lexicographical signals, but struggle when texts share similar vocabulary distributions throughout training data compared to unseen data. Conclusively, both models perform worse on unseen data compared to our validation/training texts. Perhaps the reason why our model 2 performs better suggests that its feature tunings captures more additional information. If we were to continue working on this project, we would attempt to find a more optimal contextual embeddings, which captures deeper stylistic interpretations between texts. In short, generalize our model better to everything!

AI Disclosure: We used LLMs(ChatGPT) to help generate the code to produce our flow charts, as well as generate the code to create the vector embeddings from the pretrained Word2Vec dataset.